

How devtooling resources get into LLM training data

To be trained on, your content has to clear **two gates**: get collected (a web crawl or public code hosting), then survive the corpus builder's quality filter. Crawlable does not mean trained on. Here is what to do about it, then how it works.

Research note · last updated 2026-06-21 · every major fact links a primary source

CONTENTS

- | | |
|---|--|
| 1. DO and DON'T | 5. Controlled study: what actually predicts memorization |
| 2. How it works (the short version) | 6. Key terms |
| 3. What the corpus builders say | 7. How to check |
| 4. Worked example: memorized despite failing the quality gate | 8. Caveats and what is contested |
| | 9. Sources |

DO and DON'T

The biggest lever first: **a library is learned mostly from its public code, not its website**. Code corpora skip the web quality filter entirely.

DO

- **Put code and docs in a public GitHub repo**. Code corpora ([The Stack](#)) ingest it directly, no quality filter. A permissive license (MIT/Apache) is the safe choice, and **a missing license is not a blocker either**: The Stack v2 and v3 keep files labeled `no_license` and exclude only `non_permissive` ones. (This was different in v1, which dropped everything without a detected permissive license.)
- **Invest in the README and a runnable `examples/` directory**. The README is often the single most-trained piece of text about a library.

- **Keep docs source as full-prose Markdown in that public repo.** The Markdown itself is trained on, regardless of whether the rendered site is crawled.
- **Write tutorial / explanatory blog posts**, and **lead the first ~380 words** with what the reader will learn, defining key terms in plain sentences. Only the opening is scored, and this is the lever that clears the bar (measured: 2.43 → 3.19).
- **Earn backlinks and internal links** (newsletters, aggregators, cross-posts) so Common Crawl actually reaches new and deep pages.
- **Fix crawl hygiene:** `sitemap.xml` , `301` redirects for moved URLs, expand thin pages, cut click-depth.
- **Verify** (see [How to check](#)): is it in Common Crawl, does it clear the keep bar (`int_score` ≥ 3, i.e. `raw` ≥ 2.5), are crawlers unblocked.

DON'T

- **Don't rely on the rendered docs site alone.** Reference and config docs mostly fail the quality filter (measured: AnyCable guides cap ~1.7-2.0, below the keep bar even after a full rewrite). Let the GitHub channel carry them.
- **Don't keep content in a private repo, or under a copyleft/proprietary license.** Private content never enters code corpora, and `non_permissive` files (GPL and friends, "all rights reserved") are excluded by name. An *absent* license is fine for The Stack v2 and v3 (**corrected: we previously said unlicensed repos are excluded, which was true only of v1**).
- **Don't count on stars.** Star count is **not** an inclusion filter in any version of The Stack. It only decides whether a *fork* gets crawled (≥ 5 stars) and which copy survives deduplication.
- **Don't lead posts with war-stories, anecdotes, or marketing.** Benchmark and announcement genres cap below the keep bar (measured: 1.5-1.9, which rounds to 2) no matter how you edit them.
- **Don't bury the substance below the fold.** The classifier reads only the first ~512 tokens.
- **Don't expect `llms.txt` to help.** [No major AI system uses it](#) for training or inference.
- **Don't just strip boilerplate** and expect a better score. It does not raise quality (**measured: it can lower it**); rewrite the substance instead.
- **Don't block `CCBot` , `ClaudeBot` , `GPTBot` in `robots.txt`** if you want inclusion.

How it works (the short version)

Two gates

1. **Collection.** A [web crawl](#) (Common Crawl, or a lab's own crawler) or [public code hosting](#) (GitHub).

2. **Filtering.** Builders drop most of what they collect. Modern web pipelines keep on the order of 10% of raw crawl (RefinedWeb removed ~90%).

Four channels

Channel	Feeds	Collected by	Quality gate
Code + docs-in-repos	Code corpora (The Stack)	GitHub, public; permissive or no license	none (license + dedup)
Blog posts	Web corpora	Common Crawl	rewards tutorial prose
Documentation (rendered)	Web corpora	Common Crawl	hostile to reference/code
Content website	Web corpora	Common Crawl	rewards little (marketing)

Why most devtooling content struggles

- **Common Crawl is centrality-sampled, not complete.** It ranks domains by harmonic centrality (how linked-to you are), gives higher-ranked domains a bigger crawl budget, and **never copies a domain in full** (FAccT'24). Hubs get in; deep, new, weakly linked pages mostly don't. (measured: **evilmartians.com** 37%, **docs.anycable.io** 27%; crawled and missing pages scored identically, so the gap is reach, not quality.)
- **The quality filter rewards educational prose and penalizes reference, code, and marketing.** The open FineWeb-Edu classifier (keeps a document when its score rounds to ≥ 3 , i.e. a raw score ≥ 2.5) was deliberately tuned toward school-level explanation and away from technical pages. Its ancestor C4 was blunter still: it dropped *any whole page containing a curly brace* `{` (a 2019 rule; modern filters are softer). (measured: 5 of 598 **evilmartians.com** pages and 0 of 117 **AnyCable docs** pages clear the bar.)
- **Code corpora skip that filter.** The Stack takes GitHub files (including ~254 GB of Markdown) with no prose-quality classifier, which is why the public-repo path is the strongest one. Each file is labeled `permissive`, `no_license`, or `non_permissive`, and **only `non_permissive` is dropped**.
- **Collection is its own gate, before any license test.** The Stack v2 and v3 are built from the Software Heritage archive, and archival there is neither guaranteed nor star-driven. (measured: **workos/authkit** (3.3k stars, MIT), **workos/workos-node** and **resend/resend-node** are absent from SWH, while **workos/dotnet-magic-link-example** is present.) A permissive license on an uncollected repo buys nothing.
- **Deduplication collapses near-identical copies.** MinHash-LSH clustering removed ~40% of permissively licensed files in v2, and v3 keeps a single representative per cluster (chosen by highest stars, then forks, then permissive license, then earliest repo creation). Mirroring your docs verbatim into other repos mostly feeds

dedup; **embedded reuse**, your sentences quoted inside someone else's different document, is what survives as a distinct training example.

And capability follows representation: models do well on high-resource languages and [struggle on low-resource ones](#), though [synthetic/fine-tuning data](#) can close gaps, so data share is not destiny.

What the corpus builders say

The pipelines above are documented in papers. Their creators have also described the mechanics in recent talks and interviews, and they converge on four points, each with a consequence for your content.

Raw web text is mostly thrown away. Thomas Scialom, a Llama 3 lead at Meta, puts it plainly: the web is ["full of shit in terms of text, and training on those tokens is a waste of compute"](#). Andrej Karpathy, opening a real pretraining set, calls it ["total garbage"](#). Clearing the filter is the whole game.

A learned quality classifier is the decisive filter, and humans cannot predict it. In his [DCLM keynote](#), Ludwig Schmidt reports that model-based filtering was the single most important step, and that the collaborators who hand-labeled documents "can't really tell apart" the best classifier variants. Ari Morcos [says the same](#): NLP graduate students "could not predict what the DCLM classifiers would say above chance." FineWeb-Edu's creator, Loubna Ben Allal, [describes the mechanism](#): "taking Llama3 and asking it to rate the educational content of web pages from zero to five," then training "a BERT model" to imitate it, which cuts 15T tokens to 1.5T at score > 3. You cannot eyeball whether a page clears this. Score it (see [How to check](#)).

More filtering is not always better. Over-aggressive deduplication keeps the worst material: strip too hard and ["you'll just be upsampling the worst-quality stuff"](#) (Guilherme Penedo, FineWeb). Stacking more "high-quality" sources can also hurt, and Schmidt found that adding Wikipedia and arXiv on top of already-filtered web made results worse. The educational gate keeps documents at score ≥ 3 , and pushing the threshold higher trims general-knowledge performance. Aim to clear the bar, not to maximize the score.

Educational prose helps smaller models most, and code and math are the reasoning levers. Penedo notes that heavy educational filtering ["is really important for very small models"](#), with larger models wanting more diversity later in training. Code and math are deliberately upweighted across labs for reasoning ([Jeff Dean](#), Scialom). Technical tutorial writing is exactly what the filter rewards, and the public-code channel ([The Stack](#)) is stronger still.

And the gate is tightening. As labs report exhausting fresh human text (Ilya Sutskever: the field has ["reached peak data"](#)), quality filtering and synthetic data carry more weight, so clearing the bar matters more over time.

Worked example: memorized despite failing the quality gate

We measured this directly. **(measured)** Supabase and Resend are tools the agentic-coding wave made ubiquitous, and both teams believe their docs are in training. We tested two of their stable, popular pages two ways: the FineWeb-Edu quality score, and a tools-off verbatim-recall probe against Claude Opus (give the model a page's opening and measure the longest exact continuation it reproduces; validated with known-memorized positive controls and a fabricated negative control, see `scripts/memorization_probe.rb`).

page	quality score	would the filter keep it?	verbatim recall by Opus
Supabase Auth guide	1.43	no	yes (23-word exact run)
Resend Node.js quickstart	1.68	no	yes (15-word exact run)

Both pages **fail the quality gate** (short, code-snippet quickstarts are exactly what the classifier scores low), yet Opus reproduces them verbatim. They reached training through the **code channel**, not the web one: Supabase's docs live in `supabase/supabase` (Apache-2.0, so the Markdown is in The Stack), and Resend's quickstart is the canonical snippet from its MIT-licensed `resend-node` and `resend-examples` repos, copied into ~1,840 GitHub repositories. The permissive public code bypasses the quality filter entirely, then sheer duplication tips it from "in the corpus" to "reproduced from memory."

The control sharpens it: Supabase's Row Level Security guide sits in the **same** Apache repo yet showed **no** verbatim recall, because it is prose that gets paraphrased rather than a snippet that gets copied. The lesson: eligibility comes from a public repo; memorization comes from being the canonical artifact everyone duplicates. Being in the repo is necessary; being copied is what makes a model know it by heart.

Controlled study: what actually predicts memorization

The worked example above is two cases. To test which repo property matters, we ran the probe over **38 passages from 15 repositories** (3 passages each), holding content type constant (documentation prose inside a public repo), selecting passages deterministically to avoid cherry-picking, and varying stars across five orders of magnitude (10 to 141,000) and license across MIT, Apache-2.0, and none. See `scripts/repo_probe.rb` . The unit of analysis is a repo's hit rate across its passages, not a single coin flip.

Predictor	Spearman ρ vs hit rate	p (permutation, 200k)
Independent public copies	+0.70	0.005
Forks	+0.47	0.077
Stars	+0.28	0.31 (not significant)

Stars do not predict memorization; copies do. The clearest single case: `tailwindlabs/tailwindcss.com` has **137 stars, no license, and 68 independent copies**, and it was the most-memorized repo in the sample.

`vercel/next.js` has **141,147 stars** and produced one marginal hit. Stars measure how many people liked a project. Copies measure how many times its sentences exist in the world, and only the second is what a corpus counts.

A missing license did not suppress memorization. Unlicensed repos were hit at 3/6 versus 3/32 for permissive ones (Fisher exact $p = 0.039$). We do *not* claim a missing license helps: both unlicensed repos in the sample are high-prominence docs sites, so prominence is confounded with license. The safe reading is that there is no evidence a missing license suppresses anything, which matches the documented v2/v3 policy.

It replicates across model families. Re-running the same passages against a second frontier model from a different lab gives $\rho = +0.68$ ($p = 0.0065$) on per-repo maximum recall. The same repos rank as memorized for both, which makes this a property of what is on the internet rather than of one lab's pipeline. Absolute hit rates are *not* comparable between the two runs (different output lengths); the ranking is.

One honest limit: `rails/rails` shows only 2 GitHub copies yet produced a 30-word exact run, because Rails Guides are duplicated heavily *on the web* rather than in repos. GitHub code search sees one slice of duplication, so treat the copy count as a proxy for total duplication, not a census.

Key terms

- **Centrality (harmonic centrality):** a link-popularity score; the more pages link to you, directly or via short chains, the higher it is. Crawlers use it to decide what to fetch. *Raise it with backlinks.*
- **Crawl budget:** the capped number of URLs a crawler fetches from one domain; higher centrality buys more.
- **Recall (coverage):** the share of a site's real pages actually crawled (we measured 27-37%).
- **Corpus:** a curated dataset a model trains on (C4, FineWeb, The Stack).
- **Quality classifier:** a small model that scores "educational value" and drops low scorers (FineWeb-Edu keeps $\text{int_score} \geq 3$, i.e. a raw score ≥ 2.5).
- **Token:** the unit a model reads, roughly $\frac{3}{4}$ of a word; only the first ~ 512 (~ 380 words) get scored.

- **Permissive vs. copyleft license:** code corpora label each file `permissive` (MIT, Apache), `no_license`, or `non_permissive` (GPL and friends). The first two are kept; only `non_permissive` is dropped.
- **Near-deduplication:** clustering near-identical files (MinHash-LSH) and keeping one representative, so wholesale copies of your docs collapse into a single training example.
- **The Stack:** a large public dataset of GitHub code and docs used to train code models; bypasses the web quality filter.
- **Pretraining vs. fine-tuning:** pretraining is where a model learns whether it "knows" Ruby; that is the target here. Fine-tuning shapes later behavior.
- **RAG / MCP / llms.txt:** inference-time retrieval, they affect what a model looks up at runtime, not what it was trained on.

How to check

Question	How	Tool
Is it in Common Crawl?	CDX query <code>url=domain/*</code> per crawl	index.commoncrawl.org , <code>scripts/coverage_details.rb</code>
Would the filter keep it?	Score with FineWeb-Edu (<code>int_score ≥ 3</code> , i.e. <code>raw ≥ 2.5</code>); only ~512 tokens are read	classifier , <code>scripts/quality_core.rb</code> , the <code>/check</code> page
Could the code be in The Stack?	Repo public, license not copyleft, and archived by Software Heritage	SWH API <code>origin/https://github.com/<owner>/<repo>/visit/latest/</code>
Is the text actually memorized?	Tools-off verbatim-continuation probe, with positive and negative controls	<code>scripts/memorization_probe.rb</code> , <code>scripts/repo_probe.rb</code>
Are crawlers allowed?	<code>robots.txt</code> for <code>CCBot</code> / <code>ClaudeBot</code> / <code>GPTBot</code>	server logs, <code>robots.txt</code>

Caveats and what is contested

- **Proxies, not ground truth.** FineWeb-Edu and C4 are *open, documented* filters. Frontier labs (Anthropic, OpenAI, Google) do not disclose theirs, so our scores show what representative open pipelines do, not what Claude's filter does. The one lever you control for Anthropic is `robots.txt` ([\(blocking ClaudeBot excludes future content from training\)](#)).
- **Single-sample, lead-only.** Our quality numbers are single runs; differences within ± 0.3 are noise, and only the first ~ 512 tokens are scored.
- **Era matters.** C4's curly-brace rule is from 2019; modern corpora filter differently. The same applies to licensing: The Stack **v1 excluded** unlicensed repos, **v2 and v3 include** them. Any advice about licenses has to name the corpus version it describes.
- **Inclusion $\neq$ influence.** Trained-on content is still deduplicated and weighted; presence is not learning.
- **Negative memorization results prove little.** Models memorize a small fraction of what they train on, so "no verbatim recall" is only interpretable in aggregate and against controls. Positive results are strong; absences are weak.
- **There is no working public Stack-membership oracle.** The ["Am I in the Stack?"](#) space now answers "No" for every input, including `torvalds` and `rails`, since the dataset behind it became gated. We removed it from *How to check* for that reason. Do not trust it; check collection (Software Heritage) and license separately.
- **Run memorization probes in an empty directory, with tools off.** Agent-style CLIs can read their working directory. Our first cross-model run scored a perfect 1.0 n-gram overlap on a 10-star repo because the agent found the probe's own output file, containing every expected answer, on disk. The same passage from an empty directory scored 0.0. Validate positive controls under your exact prompt and backend before believing any number, and treat a suspiciously perfect result as a bug report.

Sources

- Common Crawl: [Baack, FAccT'24](#) · [Mozilla](#) · [CC index](#)
- Web filters: [C4 / T5 \(Raffel et al.\)](#) · [Documenting C4 \(Dodge et al.\)](#) · [RefinedWeb](#) · [FineWeb / FineWeb-Edu](#) · [classifier](#)
- Code corpora: [StarCoder2 / The Stack v2](#) · [The Stack v1](#) · [The Stack v2](#) · [stack-v3-train](#) (license classes, dedup representative rule) · [Software Heritage API](#)
- Representation: [MultiPL-E](#) · [low-resource / MultiPL-T](#)
- Anthropic & llms.txt: [ClaudeBot docs](#) · [Google on llms.txt](#)
- Talks & interviews (primary, spoken): [Penedo / FineWeb, GOSIM 2025](#) · [Ben Allal / FineWeb-Edu, Latent Space @ NeurIPS 2024](#) · [Wolf / building LLMs in 2024](#) · [Schmidt / DCLM keynote 2025](#) · [Morcos / data](#)

[curation, Latent Space 2025](#) · [Scialom / Llama 3, Latent Space 2024](#) · [Karpathy, Dwarkesh 2025](#) · [Sutskever / "peak data", NeurIPS 2024](#)

Last updated 2026-06-21. Measured findings are reproducible via [scripts/quality_core.rb](#) and [scripts/coverage_details.rb](#) in this repo.

Generated from [how-devtooling-gets-into-training-data.md](#). Findings marked "(measured)" are reproducible via [scripts/quality_core.rb](#) and [scripts/coverage_details.rb](#) in this repo.